Massey University
College of Sciences

161.304: Advanced Statistical Modelling

Solutions to Assignment 1, 2024

Solution to Exercise 1: How much water on Earth
You can rely on the code in the Chapter to do most of this exercise.

1. (a) W, W, W observations:

```
p_grid <- seq(from=0 ,to=1 , length.out=1000)
prior <- rep(1 , 1000)
prob_data <- dbinom(3 , size=3 , prob=p_grid)
posterior <- prob_data*prior
posterior <- posterior/sum(posterior)
samples <- sample(p_grid, prob=posterior, size=10000,
   replace=TRUE)
library(rethinking)
dens(samples, show.HPDI = 0.9)
```

   and you simply need to replace 3 in `dbinom` by 5 and the `size=3` option by `size=7` for (b) when the data is L, W, W, L, W, W, W.

   You could have used `plot(p_grid, posterior)` to plot the posterior directly.

   And of course feel free to play with the level of the credibility interval you want to show. I have put 90%.

   You can comment on the clear effect of the data on the posterior.

2. Now if we modify the prior:

```
p_grid <- seq(from=0 ,to=1 , length.out=1000)
prior2 <- c(rep(0, 500 ), rep(1, 500))
posterior2 <- prob_data*prior2
posterio2r <- posterior2/sum(posterior2)
samples2 <- sample(p_grid, prob=posterior2, size=10000,
   replace=TRUE)
dens(samples2, show.HPDI = 0.9, xlim = c(0,1))
```

   Notice it doesn't matter too much how I define the prior up to a constant. In theory, the true prior density has level 2 on $[0.5, 1]$ to integrate to 1.

   The interesting bit is that more mass goes around the approximate true value of 70% of water on Earth.

   The data can still be quite misleading of course. We will see later in the course how informative priors can help, when there is so little data. it is used a lot in many fields.

3. One approach to this problem is to try a range of sample sizes and plot the interval width for each. An implementation:

```
N <- 2300 #I tried different values and this one looks OK
p_true <- 0.7
W <- rbinom(1, size=N, prob=p_true)
p_grid <- seq(from=0, to=1, length.out=1000)
prior <- rep(1 , 1000) #feel free to modify it
prob_data <- dbinom(W, size=N , prob=p_grid)
posterior <- prob_data*prior
posterior <- posterior/sum(posterior)
samples <- sample(p_grid ,prob=posterior ,size=10000,
   replace=TRUE )
PI99 <- PI(samples , 0.99)
as.numeric(PI99[2] - PI99[1])
```

Now be clever, make a function (of N) of this and repeat executing it a "large" number of times (100 is enough, but you can do more to be safe) to get an average behaviour. You should find our that 2000 is just enough.

The above is a general feature of learning from data: The greatest returns on learning come early on. Each additional observation contributes less and less (remember the CLT argument with $1/\sqrt{N}$ driving the width of the interval?). So it takes very much effort to progressively reduce our uncertainty. So if your application requires a very precise estimate, be prepared to collect a lot of data. Or to change your approach.

Solution to Exercise 2: Earth vs Mars
We just have to write things down, the probability of land, given earth is `p_le <- 0.3`; he probability of land, given Mars `p_lm <- 1.0`; the probability of Earth `p_e <- 0.5`.

Using the law of total probability, the probability of land is: `p_l <- (p_e*p_le)+((1-p_e)*p_lm)`, and you can derive the probability of Earth, given land (using Bayes' Theorem):

```
p_el <- (p_le*p_e)/p_l
p_el
[1] 0.231
```

Solution to Exercise 3: Panda vs Panda
After the first birth (first data point), the likelihood of species A and B is equal to the rate the species give birth to twins, either `a_likelihood <- 0.1` or `b_likelihood <- 0.2`.

Next calculate the posterior probability that the panda belongs to each species, if we assume the species are equally likely (flat prior):

```
likelihood <- c(a_likelihood, b_likelihood)
prior <- c(1, 1)
posterior <- likelihood*prior
posterior <- posterior/sum(posterior)
posterior
[1] 0.333 0.667
```

Now, the probability the next birth is twins is the probability the panda belongs to each species times the likelihood each species gives birth to twins:

```
(posterior[1]*a_likelihood)+(posterior[2]*b_likelihood)
[1] 0.167
```

Solution to Exercise 4: !Kung census predictions and modelling

1. We start by estimating the linear model (quadratic approximation of the posterior):

   ```
   data(Howell1)
   d <- Howell1
   d2 <- d[d$age>=18,]
   mod <- map(alist(
     height ~ dnorm(mu ,sigma),
     mu <- a+b*weight,
     a ~ dnorm(100, 100),
     b ~ dnorm(0, 10),
     sigma ~ dunif(0, 50)
   ), data=d2)
   ```

   Notice the extremely wide priors here. I will refine later to make something meaningful. Will be easier to work on the standardised data.

   Then produce samples from the approximate posterior. We use the convenient `extract.samples()` function:

   ```
   post <- extract.samples(mod)
   str(post)
   'data.frame': 10000 obs. of  3 variables:
    $ a    : num  118 115 115 114 112 ...
    $ b    : num  0.8 0.874 0.889 0.89 0.934 ...
    $ sigma: num  4.97 4.86 4.93 5.15 4.94 ...
    - attr(*, "source")= chr "quap posterior: 10000 samples from mod"
   ```

   Now we want to plug the weights of the table -for which we want to do predictions- into this model, and then average over the posterior to compute predictions for each individual's height. The question is ambiguous as to whether it wants μ only or rather the distribution of individual height measurements (using σ). It won't change anything for the pointwise predictions, but would for the interval: around the mean prediction (compatibility interval) or for new observations (predictive interval). I will show the harder approach, that uses σ and simulates individual heights. Also, I will not use use `link()` or `sim()`, but it's fine if you did of course. For the first individual, the code might look like this:

   ```
   y1 <- rnorm(10000, post$a+post$b*45, post$sigma)
   mean(y1)
   [1] 154.6671
   HPDI(y1,prob=0.89)
      |0.89    0.89|
   146.3180 162.4944
   ```

   And you can do the same for the three other individuals with different weights.

2. We fit the following linear model first: (I didn't normalise heights)

```
data(Howell1)
d <- Howell1
d$logW <- standardize(log(d$weight))
mm <- mean(d$logW)
mod2lin <- map(alist(
  height ~ dnorm(mu ,sigma),
  mu <- a+b*logW,
  a ~ dnorm(100, 100),
  b ~ dnorm(0, 10),
  sigma ~ dunif(0, 50)
), data=d)
plot(precis(mod2lin))
```

Notice how large `b` is, in spite of my prior. The data was enough to move away from it. I had `b` $\sim$ `dnorm(0, 1)` at first, but increased the sd to 10.

And we plot the posterior along with the data:

```
x_seq <- seq(-3.5, 3.5, length.out = 50)
mulin <- sim(mod2lin, data = list(logW = x_seq))
mulinM <- apply(mulin, 2, mean)
mulinCI <- apply(mulin, 2, PI, 0.9)
plot(d$weight, d$height, col = col.alpha(rangi2, 0.3),
  pch = 16)
lines(exp(x_seq*sd(log(d$weight)) + mean(log(d$weight))), mulinM)
shade(mulinCI, exp(x_seq*sd(log(d$weight)) + mean(log(d$weight))))
```

Honestly, not sure how much better we can do. Yes, we might be under-predicting a little bit the medium ages and over-predicting the medium-high ages. You might think of it from the volume of the human body perspective considering it's a cylinder, whose radius is proportional to its height.

For the sake of it, we could go with a spline:

```
num_knots <- 10 #feel free to play with it
knot_list <- quantile(d$logW, probs = seq(0, 1,
  length.out = num_knots))
library(splines)
B <- bs(sort(d$weight), knots=knot_list[-c(1, num_knots)],
       degree = 3, intercept = TRUE)
mod2splines <- quap(alist(
  H ~ dnorm(mu ,sigma),
  mu <- a + B %*% w,
  a ~ dnorm(100, 100),
  w ~ dnorm(0, 10),
  sigma ~ dunif(0, 50)
  ), data=list(H=d$height[sort(d$weight, index.return = T)$ix], B=B),
start = list(a= 0, w = rep(0, ncol(B))))
```

Careful, depending on initial values, sometimes you can get issues. And again we can plot the posterior predictive distribution:

```
muspl <- sim(mod2splines)
musplM <- apply(muspl, 2, mean)
```

```
musplCI <- apply(muspl, 2, PI, 0.9)
plot(d$weight, d$height, col = col.alpha(rangi2, 0.3),
  pch = 16)
lines(sort(d$weight), musplM)
shade(musplCI, sort(d$weight))
```

Honestly, is the fit any better? You decide!

3. I recall the code from polynomial of order 2 in the Chapter:

```
library(rethinking)
data(Howell1)
d <- Howell1
d$weight_s <- (d$weight - mean(d$weight))/sd(d$weight)
d$weight_s2 <- d$weight_s^2
m4.5 <- quap(alist(
  height ~ dnorm(mu, sigma) ,
  mu <- a + b1*weight_s + b2*weight_s2,
  a ~ dnorm(178, 20),
  b1 ~ dlnorm(0, 1),
  b2 ~ dnorm(0, 1),
  sigma ~ dunif(0, 50)
), data=d)
```

And now we can extract the prior:

```
set.seed(2022)
prior <- extract.prior(m4.5)
precis(prior)
```

And we now simulate curves (parabolas) from these priors. Let's use `link()` so we don't have to write the linear model again:

```
w_seq <- seq(min(d$weight_s), to = max(d$weight_s),
  length.out = 50)
w2_seq <- w_seq^2
mu <- link(m4.5, post = prior, data =
  list(weight_s = w_seq, weight_s2 = w2_seq))
```

`mu` has got 1000 parabolas. We plot the first 50:

```
plot(NULL, xlim = range(w_seq), ylim = c(55,270),
  xlab = "standardised weight", ylab = "height")
for(i in 1:50){
  lines(w_seq, mu[i,], col = col.alpha("black", 0.5))}
```

The prior curvature is clearly quite weak. the parabolas hardly bend at all. We can increase the sd on `b2` (say `b2` $\sim$ `dnorm(0,10)`) but this will produce silly shapes with average weights either tallest or shortest. `b2` needs to be non-positive to have the curve bend down. But we must control `b1` to move the maximum height the largest values of the standardised weight. One idea is to write `mu <- a+b1*weight_s + b2*weight_s2` in the definition of the model and but priors on `b1` and `b2` such that:

```
b1 ~ dlnorm(2, 1),
b2 ~ dlnorm(1, 1)
```

No easy to work with polynomials. And this is order 2 only, imagine controlling the curvatures with a polynomial of order 4?! A key challenge is that it is obvious that `a`, `b1` and `b2` are correlated in the family of reasonable curves. But the priors we defined aren't correlated. Yet, if you can get independent priors to live within some reasonable space of outcome values, that's better than flat priors.

To finish, visualise flat priors that give birth to strongly explosive curves?!

```
b1 ~ dnorm(0, 100),
b2 ~ dnorm(0, 100)
```

The issue is that "flat" doesn't mean no relationship, once the prediction is complicated.

Try doing the inference with the different priors above. Does it make any difference? Well, no, because there is a lot of data and the model is quite simple in terms of the way parameters related to predictions. This is not always the case, you have been warned...

Solution to Exercise 5: "The Gamblers"

Let X_t be Algernon's fortune after t games. Then X_t is a Markov chain with state space $\{0, 1, 2, 3, 4, 5, 6\}$ and transition matrix

$$P = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0.55 & 0 & 0.45 & 0 & 0 & 0 & 0 \\ 0 & 0.55 & 0 & 0.45 & 0 & 0 & 0 \\ 0 & 0 & 0.55 & 0 & 0.45 & 0 & 0 \\ 0 & 0 & 0 & 0.55 & 0 & 0.45 & 0 \\ 0 & 0 & 0 & 0 & 0.55 & 0 & 0.45 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

and initial state $X_0 = 3$.

For the simulation in R, it is a little more convenient to use the change $Y_t = X_t + 1$ which has state space $\{1, 2, 3, 4, 5, 6, 7\}$ and initial state $Y_0 = 4$ (so that the value of Y_t is Algernon's fortune plus one dollar).

The R snippet below simulates this Markov chain for $n = 25$ games in the inner loop. A total of $N = 100,000$ replications of the simulation are performed (as the outer loop). In each case we need retain only the final state. If the final state is 1 then it means that Algernon has lost all his money. If the final state is 7 it means that Algernon has won all of Bethany's money.

```
P <- matrix(c(1,0,0,0,0,0,0,
  0.55,0,0.45,0,0,0,0,
  0,0.55,0,0.45,0,0,0,
  0,0,0.55,0,0.45,0,0,
  0,0,0,0.55,0,0.45,0,
  0,0,0,0,0.55,0,0.45,
  0,0,0,0,0,0,1),ncol=7,byrow=T)
results <- numeric(100000)
set.seed(451)
for ( i in 1:100000 ){
  y <- numeric(26)
  y[1] <- 4
  for (tt in 1:25){
  y[tt+1] <- sample(1:7,size=1,prob=P[y[tt],])
  }
```

```
  results[i] <- y[26]
}
tabulate(results)
[1] 62534     0  1560     0  1325     0 34581
gamble.props <- tabulate(results)/100000
B.p <- gamble.props[1]
A.p <- gamble.props[7]
mc.se.B <- sqrt(B.p*(1-B.p))/sqrt(100000)
mc.se.A <- sqrt(A.p*(1-A.p))/sqrt(100000)
A.p
[1] 0.34581
mc.se.A
[1] 0.001504079
B.p
[1] 0.62534
mc.se.B
[1] 0.001530653
```

From this we see that the probability of Algernon winning all the money is estimated as 0.3458 with Monte Carlo standard error 0.0015, and the probability that Bethany wins all the money is 0.6253 with Monte Carlo standard error 0.0015. Note that 25 games is sufficient to ensure that one player or the other goes broke in most of the 100,000 replications. In less than 3% of cases (2885/100000) both players still had money at the end of that sequence.

While you were asked to solve this question using stochastic simulation, you may have noticed that a direct computational solution is also possible. Specifically, we can look at the probabilities at time $t = 25$ using the computation

$$\pi^{(25)} = \pi^{(0)} P^{25}$$

where $\pi^{(0)} = (0, 0, 0, 1, 0, 0, 0)$. This can be done in R as follows:

```
pi0 <- t(c(0,0,0,1,0,0,0))
library(expm)
P25 <- P%^%25
pi0%*%P25
          [,1] [,2]        [,3] [,4]        [,5] [,6]       [,7]
[1,] 0.6279752    0 0.01544259    0 0.01263485    0 0.3439474
```

From this we see that the probability that Bethany wins all the money by time $t = 25$ is $\pi_0^{(25)} = 0.6280$. Our Monte Carlo estimate was pretty accurate.